

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code:	CSA10 / CSA1023	Course Title:	Software Engineering
CO Assessed:	CO1	Assessment:	Assessment Tool 1 – Case Study Analysis
Weightage:	20%	Total Marks:	25
CO1 Statement:	Apply advanced methodologies and agile project management techniques to manage software projects effectively		
Student Name:	C.Devasri	Reg. No.:	192521422
Date:	7/10/2026	Duration:	60 minutes

Code of Conduct

I C.Devasri (192521422) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: C.Devasri

Annexure A – Case Study Analysis

Instructions to Students:

Each student will be allotted **one problem statement**. Analyze the assigned problem systematically by following the steps given below. Your report should demonstrate your understanding of software engineering concepts and your ability to design an appropriate software solution.

Based on your allotted problem statement, prepare a report by answering the following questions:

- 1. Understand the Problem Statement**
 - Explain the problem in your own words.
 - Identify the objectives of the proposed system.
 - State the scope of the problem.
- 2. Identify Key Stakeholders and Challenges**
 - Identify all the stakeholders involved in the system.
 - Explain the roles and responsibilities of each stakeholder.
 - Discuss the major challenges and issues faced by the stakeholders.
- 3. Analyze the Current Approach**
 - Describe the existing system or current process.
 - Identify its strengths and limitations.
 - Analyze the problems or gaps that need to be addressed.
- 4. Propose a Suitable Solution Using Software Engineering Principles**
 - Design a software-based solution for the given problem.
 - Describe the major functional modules of the proposed system.
 - Identify the functional and non-functional requirements.
 - Explain how software engineering principles such as modularity, scalability, maintainability, reliability, usability, and security are incorporated into your solution.
- 5. Justify the Chosen Methodology/Framework**

- Select an appropriate Software Development Life Cycle (SDLC) model or software development framework (e.g., Agile, Scrum, Waterfall, Spiral, DevOps).
- Justify why the selected methodology/framework is suitable for the proposed system.
- Explain how it supports successful project development and maintenance.

6. Discuss Expected Outcomes and Limitations

- Describe the expected benefits of the proposed solution.
- Explain how the solution addresses the identified challenges.
- Discuss potential risks, implementation challenges, and limitations.
- Suggest possible future enhancements.

CASE STUDY ANSWER

Problem 8: Privacy-Preserving Smart Campus Digital Twin

1. Understanding the Problem Statement

1.1 Problem in Own Words

A modern campus has many separate systems: electricity meters, air-conditioning, classroom and room bookings, shuttle transport, security and access control, and student services. Each system produces data, but the data sits in silos, so managers cannot see the whole campus in real time. Simply collecting everything in one central database would create a new problem: it exposes **personally identifiable information (PII)** such as who entered which room, when, and where students travel. This raises privacy, security, scalability and ethical concerns.

The task is to design a **Digital Twin**, a live virtual copy of campus operations, that gives managers operational visibility and helps cut energy use, while **collecting as little personal data as possible**, protecting what is collected, and keeping a record of who accessed it.

1.2 Objectives of the Proposed System

No.	Objective	Measurable Target
O1	Integrate real-time data from campus facilities and services (energy, HVAC, occupancy, transport, access)	Live data from at least 5 source types, latency below 5 seconds
O2	Provide digital-twin dashboards for operational monitoring	Role-based dashboards for building, floor and room level
O3	Use a scalable, event-driven, API-based architecture	Handle 10,000+ events per second; horizontal scaling
O4	Minimize collection and exposure of personal information	Zero raw PII stored in the twin; only pseudonymized or aggregated data
O5	Implement access control, audit trails, testing and CI/CD	100% of data access logged; automated pipeline on every commit
O6	Monitor system performance and resource consumption	Live CPU, memory and latency metrics with alerts
O7	Support sustainable campus operations	10-15% reduction in energy waste through optimization

1.3 Scope of the Problem

Aspect	In Scope	Out of Scope
Data	Energy, HVAC, occupancy counts, transport and booking events (selected, anonymized)	Facial images, personal location tracking, personal academic or health records
Features	Event streaming, REST APIs, dashboards, RBAC, audit trail, retention policy, monitoring, CI/CD	Replacement of existing campus ERP/LMS systems

Aspect	In Scope	Out of Scope
Users	Facility, energy, security, IT, privacy and management roles	Public (external) access
Deployment	Containerized (Docker) on Kubernetes, on-premise or private cloud	Hardware sensor manufacturing
Phase	Pilot on selected buildings, then scale campus-wide	Multi-campus federation (future work)

2. Key Stakeholders and Challenges

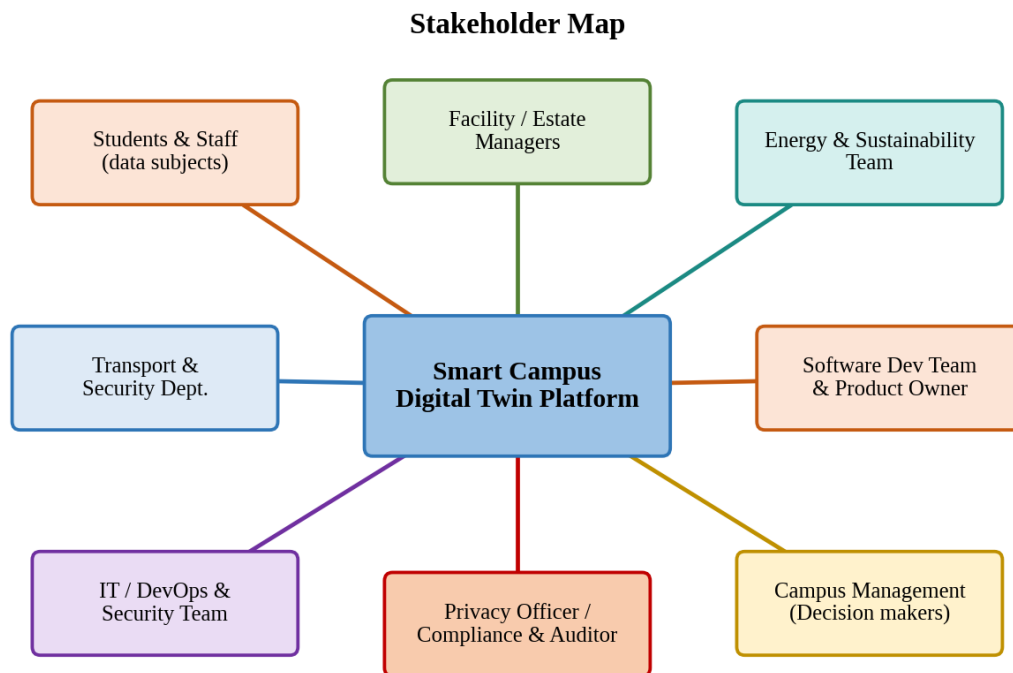


Figure 1: Stakeholder map of the Smart Campus Digital Twin

2.1 Stakeholders, Roles and Responsibilities

Stakeholder	Role / Responsibility	Main Interest
Students and Staff	Data subjects whose activity produces data; give consent; use campus services	Privacy, transparency, no tracking
Facility / Estate Managers	Monitor buildings and rooms; respond to alerts; schedule maintenance	Real-time visibility, fewer breakdowns
Energy and Sustainability Team	Analyze consumption; apply optimization rules; report savings	Lower energy cost and carbon footprint
Campus Management	Approve budget; use dashboards and reports for decisions	Cost savings, reputation, compliance
IT / DevOps and Security Team	Deploy, monitor and secure the platform; manage CI/CD and access	Uptime, security, performance

Stakeholder	Role / Responsibility	Main Interest
Privacy Officer / Auditor	Define retention and anonymization policy; review audit logs	Legal compliance, accountability
Transport and Security Dept.	Supply shuttle and access events; act on operational alerts	Reliable data, safe operations
Software Dev Team and Product Owner	Analyze requirements, develop, test and maintain the system	Clear requirements, maintainable code

2.2 Major Challenges Faced

Challenge	Description	Affected Stakeholders
Privacy of personal data	Access logs, Wi-Fi and location data can identify individuals and reveal their habits	Students, staff, Privacy Officer
Data silos and heterogeneity	Different formats and protocols (MQTT, CSV, vendor APIs) make integration hard	IT team, developers
Security threats	Central data store and IoT devices are targets for breaches and unauthorized access	IT team, management
Scalability	Thousands of sensors producing continuous real-time streams	IT team, facility managers
Ethical and legal compliance	Must follow data-protection principles (consent, purpose limitation, retention)	Privacy Officer, management
Energy inefficiency	No unified view, so lights and HVAC run in empty rooms	Energy team, management
Trust and adoption	Users may reject a system they see as surveillance	Students, staff

3. Analysis of the Current Approach

3.1 Existing System / Current Process

Most campuses today run **separate, standalone systems**: a Building Management System (BMS) for HVAC, a separate energy meter portal, a standalone access-control database, a booking tool, and a transport tracker. Data is checked manually or exported as periodic reports, and raw data (including card IDs and user names) is often stored in full and indefinitely, with little formal audit.

3.2 Strengths and Limitations

Strengths	Limitations
Each system is mature and works well for its own function	Data silos: no single campus-wide view
Low change effort, since staff are already trained	Mostly batch or manual reporting, so decisions are delayed
Vendor-supported hardware and software	Raw PII collected and stored longer than needed

Strengths	Limitations
Simple, isolated systems are easy to understand	Weak or inconsistent access control and no unified audit trail
Some local security controls exist	Monolithic or vendor-locked systems that scale poorly
	No automated testing or deployment; changes are slow and risky
	No central monitoring of performance or resource use

3.3 Gaps to Be Addressed

Gap	Current State	Required Improvement
G1	Isolated data sources	Unified, event-driven integration layer
G2	Delayed, manual insight	Real-time digital-twin dashboards and alerts
G3	Excessive personal data collected	Privacy-by-design: minimize, anonymize, aggregate
G4	Weak access control and audit	RBAC, encryption and immutable audit logs
G5	Unlimited data retention	Automated retention and deletion policies
G6	Manual deployment, no testing	Automated testing and CI/CD pipeline
G7	No resource visibility	Monitoring of performance and resource consumption
G8	Wasted energy	Analytics-driven energy optimization

4. Proposed Solution Using Software Engineering Principles

4.1 Solution Overview

The proposed solution is a **layered, event-driven, microservice-based platform**. Campus data sources publish events through an **Edge Privacy Gateway**, which removes or pseudonymizes PII before data enters the system. Events flow through a streaming backbone to independent microservices exposed through a secured API gateway, and are shown on role-based digital-twin dashboards. The whole system runs in containers with automated CI/CD and continuous monitoring.

Privacy-Preserving Smart Campus Digital Twin - Layered Architecture

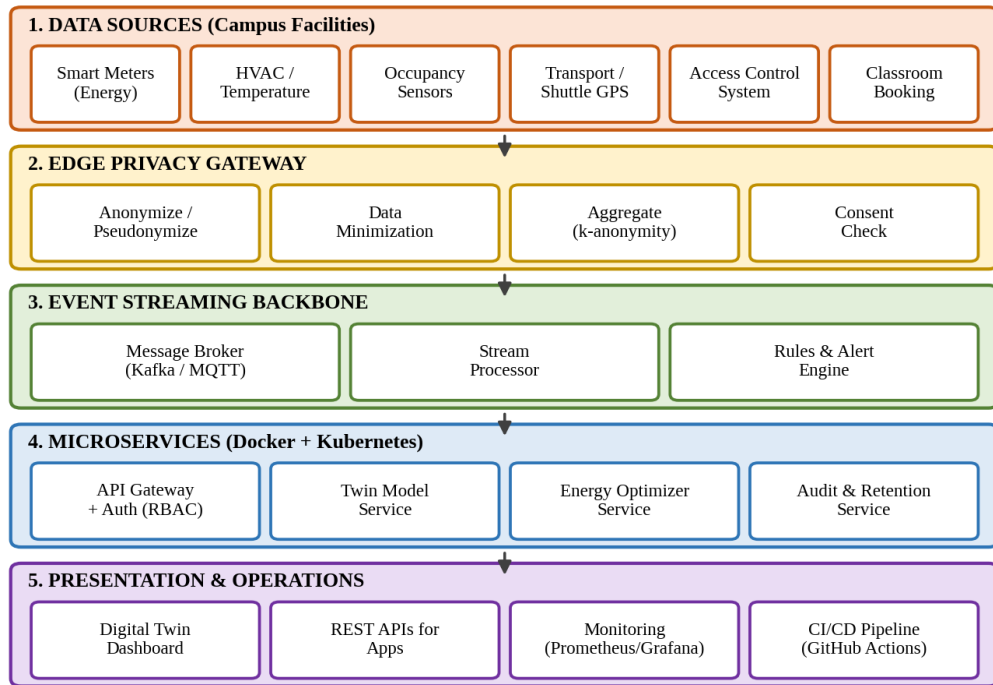
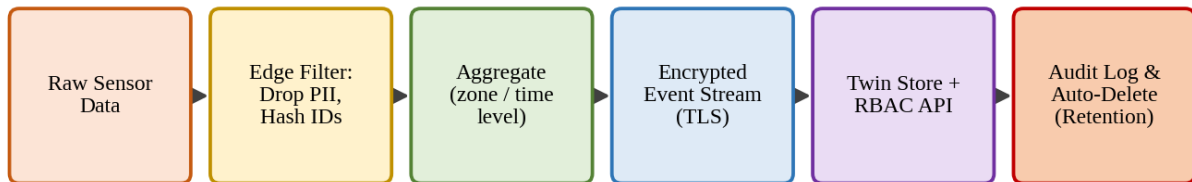


Figure 2: Layered architecture of the Privacy-Preserving Smart Campus Digital Twin

Privacy-by-Design Data Flow



Principle: collect less -> anonymize early -> restrict access -> log everything -> delete on schedule

Figure 3: Privacy-by-design data flow from sensor to deletion

4.2 Major Functional Modules

Module	Function	Technology (Suggested)
M1: Data Ingestion	Connect to sensors and campus systems; receive events in real time	MQTT, REST adapters
M2: Privacy Gateway	Remove PII, hash IDs, aggregate to zone or time level, check consent	Python / Node service
M3: Event Streaming	Reliable, scalable message delivery and stream processing	Apache Kafka

Module	Function	Technology (Suggested)
M4: Digital Twin Service	Maintain live virtual model of buildings, rooms and equipment	Microservice + time-series DB
M5: Energy Optimizer	Rules and analytics to switch off or adjust HVAC and lighting, detect anomalies	Python analytics
M6: API Gateway and IAM	Single secure entry, authentication, role-based access control	OAuth2 / JWT, RBAC
M7: Dashboard	Visual monitoring by role: building, floor, room, energy	React + charts
M8: Audit and Retention	Immutable access logs; scheduled deletion of expired data	Append-only log store
M9: Monitoring	Track CPU, memory, latency, errors; raise alerts	Prometheus + Grafana
M10: CI/CD and Deployment	Automated build, test, scan, containerize and deploy	GitHub Actions, Docker, Kubernetes

4.3 Functional Requirements

ID	Requirement
FR1	The system shall ingest real-time data from energy, HVAC, occupancy, transport and access sources.
FR2	The system shall anonymize or pseudonymize personal identifiers before storage.
FR3	The system shall display a live digital-twin dashboard for buildings, floors and rooms.
FR4	The system shall provide REST APIs for authorized applications to read twin data.
FR5	The system shall authenticate users and enforce role-based access control.
FR6	The system shall record every data access and configuration change in an audit log.
FR7	The system shall automatically delete or aggregate data when its retention period expires.
FR8	The system shall recommend or trigger energy-saving actions (e.g., turn off HVAC in empty rooms).
FR9	The system shall generate alerts for faults, anomalies and threshold breaches.
FR10	The system shall expose system health and resource metrics for monitoring.

4.4 Non-Functional Requirements

Attribute	Requirement	Measure
Performance	Dashboard updates quickly after an event	Latency < 5 s; API response < 500 ms
Scalability	Add sensors and services without redesign	Horizontal scaling; 10,000+ events/s
Availability and Reliability	Continue to operate despite component failure	99.5% uptime; retry and failover
Security	Encrypted transport and storage; least privilege	TLS 1.2+, AES-256, RBAC

Attribute	Requirement	Measure
Privacy	No raw PII stored in the twin; purpose limitation	0 raw identifiers; retention enforced
Usability	Simple, role-specific dashboards	New user productive within 30 minutes
Maintainability	Modular, documented, well-tested code	Test coverage above 80%
Portability	Run on any container platform	Docker / Kubernetes
Auditability	Tamper-evident record of access	100% of access events logged

4.5 Software Engineering Principles Applied

Principle	How It Is Incorporated	Benefit
Modularity	Separate microservices (ingestion, privacy, twin, optimizer, audit) with clear APIs	Independent development and testing
Scalability	Kafka streaming, stateless services, Kubernetes auto-scaling	Handles growth in sensors and users
Maintainability	Clean code, documentation, version control, automated tests, loose coupling	Easy to fix and extend
Reliability	Message replay, retries, health checks, redundancy, self-healing containers	Fewer outages and no data loss
Usability	Role-based dashboards, simple visuals, consistent UI	Faster decisions, higher adoption
Security	RBAC, encryption, secure API gateway, vulnerability scans in the pipeline	Protects data and system
Privacy by Design	Data minimization, anonymization at the edge, retention limits, audit trail	Responsible and ethical data use
Separation of Concerns	Layered architecture (source, privacy, stream, service, presentation)	Clear responsibilities and easy change

5. Justification of the Chosen Methodology / Framework

5.1 Selected Framework: Agile (Scrum) with DevOps (CI/CD)

The project uses **Agile Scrum** to manage requirements and delivery in short sprints, combined with a **DevOps CI/CD pipeline** for automated build, test, security scan, container packaging and deployment.

Agile (Scrum) + DevOps CI/CD Delivery Cycle

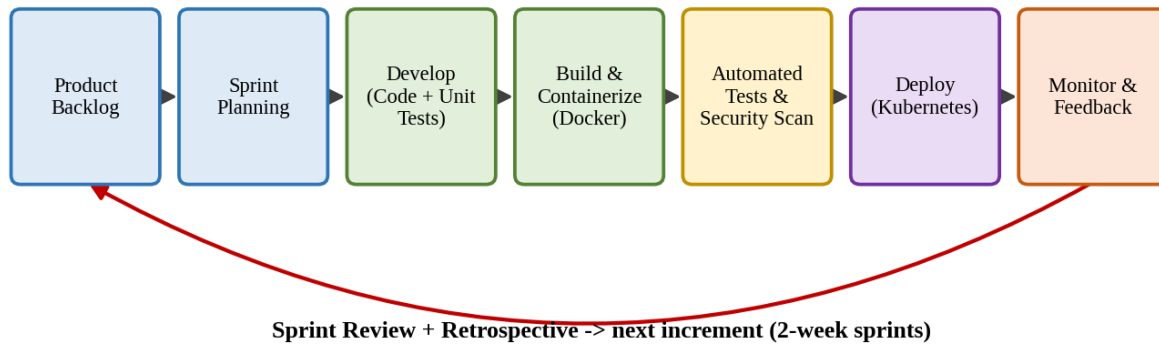


Figure 4: Agile (Scrum) with DevOps CI/CD delivery cycle

5.2 Why It Is Suitable

Project Need	How Agile + DevOps Meets It	Why Others Fall Short
Changing and unclear requirements (new sensors, privacy rules)	Backlog is reprioritized every sprint; change is welcome	Waterfall cannot absorb change
Multiple stakeholders with different needs	Sprint reviews give regular feedback and early demos	Waterfall shows results only at the end
Microservices that evolve independently	Small teams deliver increments of each service	Heavy upfront design in Spiral/Waterfall is slow
Required CI/CD, testing and containerization	DevOps automates build, test and deploy on every commit	Traditional models lack automation
Security and privacy risk	Early and continuous testing; security checks in the pipeline (shift-left)	Late testing finds privacy flaws too late
Need to show value quickly	Working pilot (one building) in first sprints	Single big-bang release delays value

Comparison summary: Waterfall is too rigid, Spiral is heavy for a team-sized project, and pure Agile without DevOps would not meet the automated delivery and monitoring requirements. Scrum plus DevOps best fits this project.

5.3 Support for Development and Maintenance

Phase	Support Provided
Development	2-week sprints, sprint backlog, daily stand-ups, definition of done, pair review, continuous integration
Testing	Unit, integration, API, security and privacy tests run automatically in the pipeline
Deployment	Docker images deployed to Kubernetes with rolling updates and quick rollback

Phase	Support Provided
Maintenance	Monitoring feedback goes back to the backlog; small frequent patches; retrospectives improve the process

5.4 Sample Sprint Plan

Sprint	Goal	Deliverables
Sprint 1	Foundation	Requirements, architecture, CI/CD skeleton, Docker setup
Sprint 2	Ingestion and privacy	Sensor adapters, privacy gateway, Kafka streaming
Sprint 3	Twin and API	Twin service, REST APIs, RBAC authentication
Sprint 4	Dashboard and audit	Dashboards, audit log, retention job
Sprint 5	Optimization and monitoring	Energy optimizer, Prometheus/Grafana monitoring
Sprint 6	Hardening and release	Security and load testing, pilot deployment, documentation

6. Expected Outcomes and Limitations

6.1 Expected Benefits

- **Improved visibility:** one real-time view of the whole campus.
- **Data-driven facility management:** faster fault detection and predictive maintenance.
- **Reduced personal data collection:** only anonymized or aggregated data is stored.
- **Energy and resource efficiency:** less waste, lower bills and carbon footprint.
- **Scalable and maintainable services:** new sensors and services can be added easily.
- **Stronger privacy and trust:** audit trails and retention build accountability.

6.2 How the Solution Addresses the Challenges

Challenge	Solution Feature
Privacy of personal data	Edge Privacy Gateway: PII removal, hashing, aggregation, consent check
Data silos	Event-driven integration layer with standard adapters and APIs
Security threats	RBAC, TLS and encryption, API gateway, pipeline security scans
Scalability	Kafka, stateless microservices, Kubernetes auto-scaling
Compliance and ethics	Audit trail, retention policy, purpose limitation, transparency
Energy inefficiency	Energy Optimizer module and occupancy-based control
Trust and adoption	Privacy-by-design, clear communication, role-based and simple dashboards

6.3 Risks, Implementation Challenges and Limitations

Risk / Limitation	Description	Mitigation
Re-identification risk	Aggregated data may still reveal individuals in small groups	k-anonymity, minimum group size, noise
Sensor or data quality	Faulty or missing sensor data lowers accuracy	Validation, outlier detection, redundancy
Integration difficulty	Legacy and vendor-locked systems are hard to connect	Adapters and gateways; phased integration
Cost and skills	Needs infrastructure and skilled staff	Pilot first; open-source tools; training
Security breach	Platform is a high-value target	Least privilege, patching, monitoring, incident plan
User resistance	Fear of surveillance	Transparency, consent, privacy notice
Performance overhead	Privacy processing adds latency	Edge processing, caching, load testing
Limited scope	Pilot covers selected buildings only	Incremental rollout

6.4 Future Enhancements

- **Federated learning and differential privacy** to train AI models without sharing raw data.
- **AI-based predictive maintenance and energy forecasting.**
- **Blockchain-backed audit logs** for stronger tamper-proof evidence.
- **Integration of renewable energy** (solar, battery) for smart load balancing.
- **3D / AR digital-twin visualization** and a mobile app for facility staff.
- **Multi-campus federation** with shared analytics and local data control.

6.5 Conclusion

The proposed Privacy-Preserving Smart Campus Digital Twin combines event-driven architecture, privacy-by-design and Agile-DevOps delivery to give campus managers real-time insight and energy savings while keeping personal data minimal, secure and auditable. It meets every requirement in the problem statement and applies core software engineering principles throughout.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Understanding (30%)	Clearly explains the problem, objectives, and scope with complete understanding.	Explains the problem with minor omissions.	Basic understanding with limited explanation.	Poor understanding; objectives and scope are unclear or missing.
Stakeholder & Challenge Analysis (25%)	Identifies all stakeholders, their roles, and challenges with clear analysis.	Identifies most stakeholders and challenges with adequate analysis.	Identifies only a few stakeholders or challenges.	Stakeholders and system analysis are incomplete or inaccurate.
Current System Analysis (20%)	Thoroughly analyzes the existing system, strengths, weaknesses, and identifies all gaps.	Good analysis with most strengths and weaknesses identified.	Limited analysis with partial identification of issues.	Proposed solution is unclear or inappropriate; requirements are largely missing.
Proposed Software Solution & Software Engineering Principles (15%)	Proposes an innovative, feasible solution applying appropriate software engineering principles (modularity, scalability, maintainability, security, etc.).	Suitable solution with good application of software engineering principles.	Basic solution with limited application of software engineering principles.	Methodology is inappropriate, poorly justified, or not discussed.
Methodology/ Framework Justification (10%)	Selects the most suitable SDLC/model/framework and provides strong technical justification.	Suitable methodology selected with adequate justification.	Methodology selected but justification is weak.	Outcomes, limitations, or future enhancements are missing; report lacks organization and clarity.

Criteria	Mark
Problem Understanding (30%)	/5
Stakeholder & Challenge Analysis (25%)	/5
Current System Analysis (20%)	/5
Proposed Software Solution & Software Engineering Principles (15%)	/5
Methodology/ Framework Justification (10%)	/5
TOTAL	/25